AGENTES INTELIGENTES PARA OTHELLO

Relatório técnico

GCC128 - Inteligência Artificial

Projeto Othello Agent Search

11 de novembro de 2026

Resumo

Este trabalho implementa agentes para Othello em Python, com um motor de regras independente, ferramentas para consultar e alterar o estado do jogo, uma estratégia heurística e um agente de linguagem orquestrado por LangGraph. A interface Tkinter permite acompanhar partidas entre uma pessoa e um agente ou entre agentes. O sistema também executa séries experimentais com alternância de cores e exporta métricas.

Na comparação clássica registrada, o agente heurístico venceu 39 de 40 partidas contra o agente aleatório. Esse resultado caracteriza apenas essa amostra e esse adversário. No fechamento deste relatório, ainda não havia uma execução válida de LLM × heurístico registrada no diretório de resultados compartilhado. Uma tentativa anterior foi interrompida pelo limite de tokens do provedor antes do primeiro turno; portanto, não é possível atribuir desempenho à LLM com os dados disponíveis.

1. Objetivo e escopo

O objetivo é construir um agente que perceba o estado do Othello, consulte ferramentas do ambiente, avalie alternativas, escolha e execute uma ação, repetindo o ciclo até o término da partida. A LLM participa da política de decisão, em vez de receber apenas o tabuleiro para devolver uma coordenada.

O projeto cobre as regras do tabuleiro 8×8, captura de peças, passes obrigatórios, fim de jogo, memória da partida, interface, agentes de referência e execução experimental. Minimax, Alpha-Beta e Monte Carlo não foram implementados: o enunciado os apresenta como técnicas possíveis, não como obrigação.

2. Arquitetura

O código separa regras, avaliação, agentes, ferramentas, orquestração, interface e experimentos. O motor do jogo não depende de LangChain, LangGraph ou Tkinter.

Módulo	Responsabilidade
`game.py`	Tabuleiro imutável, regras, jogadas, passes, placar e histórico
`evaluation.py`	Perfis heurísticos e ordenação de jogadas
`tools.py`	Ferramentas validadas e registro de chamadas
`agents.py`	Agentes aleatório e heurístico
`llm_agent.py`	Grafo de decisão com LangGraph
`matches.py`	Execução de partidas individuais
`experiments.py`	Séries de partidas, métricas e exportação JSON/CSV
`gui.py`	Interface Tkinter e acompanhamento da partida

O fluxo da LLM é **observar** → **avaliar alternativas** → **decidir** → **executar** → **verificar**. A aplicação avalia as jogadas nos três perfis disponíveis. Em uma chamada, a LLM escolhe o perfil e uma jogada legal, com uma justificativa breve. O ambiente valida e executa a ação; em seguida, o grafo consulta novamente o estado.

3. Regras, estado e ferramentas

O tabuleiro é representado por 64 valores: preto `1`, branco `-1` e vazio `0`. Preto inicia. Uma jogada é legal quando captura ao menos uma peça adversária numa das oito direções. Todas as peças capturadas são invertidas. Se o jogador não tem jogadas legais, deve passar; a partida termina quando nenhum dos dois pode jogar.

O histórico guarda a sequência de ações, passes, jogadores, posições, explicações e placares. O estado completo é fornecido ao modelo em cada turno, preservando a memória da partida, inclusive as jogadas do adversário.

Ferramenta	Uso
`get_board_state`	Consultar casas, jogador da vez e histórico
`get_valid_moves`	Listar as jogadas legais
`evaluate_moves`	Calcular a pontuação das alternativas por perfil
`make_move`	Validar e executar uma jogada ou passe
`get_game_status`	Consultar placar, passe obrigatório, término e vencedor

As ferramentas validam coordenadas, regras e turno. Cada chamada registra argumentos, resultado ou erro e duração. Uma instância de ferramenta só pode executar uma ação no turno em que foi criada.

4. Estratégia e decisão

A avaliação considera diferenças entre o jogador da vez e o adversário em quatro componentes: cantos ocupados, mobilidade, quantidade de peças e risco nas casas adjacentes a cantos vazios. Os pesos são:

Perfil	Cantos	Mobilidade	Peças	Risco perto de canto vazio
Balanced	30	5	1	-10
Mobility	25	10	1	-10
Corners	50	4	1	-15

O agente heurístico maximiza o perfil `balanced`, com desempate pseudoaleatório controlado por seed. O agente aleatório escolhe uniformemente entre as jogadas legais. A LLM vê as alternativas dos três perfis, escolhe um perfil e uma jogada e fornece uma explicação curta. Uma jogada inválida ou resposta fora do formato permite correções; falhas persistentes interrompem o turno sem avançar o jogo com uma jogada substituta.

5. Interface e modos de jogo

A interface Tkinter apresenta o tabuleiro, destaca jogadas legais, mostra o jogador da vez e o placar e mantém o histórico visível. A seleção dos participantes permite Humano × Agente e Agente × Agente, incluindo LLM × heurístico e LLM × LLM. As chamadas de modelo ocorrem em segundo plano. Uma partida pode ser pausada, reiniciada e exportada em JSON com histórico e métricas.

6. Método experimental

O executor alterna as cores entre partidas para reduzir o efeito da vantagem de iniciar. Os agentes clássicos recebem geradores pseudoaleatórios independentes por cor. O sistema salva dados brutos de cada partida e resumos JSON e CSV. Registra vitórias, empates, derrotas, taxa de vitória, pontos de torneio, jogadas, turnos, tempo médio por turno, chamadas e latência da LLM, além de chamadas às ferramentas.

A avaliação clássica registrada compara 40 partidas de heurístico × aleatório, com 20 partidas por cor, seeds base 42 a 61. A pontuação de torneio usada é vitória = 3, empate = 1 e derrota = 0. O comando reproduzível foi:

```
othello experiment --a heuristic --b random --games 40 --seed 42 --output artifacts/baseline
```

7. Resultados

Agente	Vitórias	Empates	Derrotas	Taxa de vitória	Pontos	Tempo médio por turno
Heurístico	39	0	1	97,5%	117	3,170 ms
Aleatório	1	0	39	2,5%	3	0,194 ms

As 40 partidas terminaram sem falhas. O heurístico realizou 1230 jogadas e um passe; o aleatório realizou 1165 jogadas e 65 passes. Foram registradas, respectivamente, 3692 e 2460 chamadas às ferramentas. Nenhuma chamada à LLM ocorreu nessa comparação.

O resultado é compatível com a vantagem de uma avaliação posicional sobre uma política aleatória nessa amostra. Ele não identifica qual componente da heurística causou a diferença: seria necessário testar ablações dos pesos. Também não demonstra desempenho contra agentes fortes ou em torneio.

Experimento com LLM

O arquivo disponível no fechamento registra uma tentativa LLM × heurístico que não iniciou nenhum turno: as duas partidas foram interrompidas antes da primeira jogada por limite de tokens do provedor. Essa tentativa não conta como resultado de desempenho. A avaliação com o modelo escolhido precisa ser concluída e os dados válidos incorporados para comparar a LLM com a heurística.

8. Limitações

- A evidência quantitativa de desempenho é de agentes clássicos; a comparação válida com LLM ainda precisa ser incorporada.
- O benchmark clássico usa um adversário aleatório, que é uma referência fraca.
- Os perfis usam pesos fixos e avaliam um sucessor por jogada, sem busca em profundidade.
- A LLM depende de disponibilidade, latência, limites de tokens e formato de resposta do provedor.
- Enviar o histórico completo em cada turno aumenta o contexto e o consumo de tokens ao longo da partida.
- Tempos dependem do hardware e do serviço utilizado.
- A interface não oferece retomada de uma partida exportada.

9. Conclusão

O projeto implementa um ambiente funcional de Othello, ferramentas validadas, memória de partida, agentes de referência, um fluxo de decisão com LLM, interface interativa e infraestrutura experimental. A avaliação clássica mostra uma diferença expressiva entre a heurística e o agente aleatório nas 40 partidas registradas. Ainda não há base para concluir sobre a força do agente LLM: é necessário concluir a execução com o novo modelo, registrar partidas válidas e discutir suas decisões, latências e falhas.

Referências do projeto

- Enunciado: *Trabalho Prático 05 - Agentes Inteligentes para Jogos Othello*, GCC128 - Inteligência Artificial.
- Implementação e instruções para execução: `README.md` e módulos em `src/othello/`.
- Dados do benchmark clássico: `docs/results/baseline.json`.